

A Frozen-DINOv3 Query-Based Pipeline for Variable-Cardinality Palette Prediction

Project Report

July 28, 2026

Abstract

This document describes the implemented supervised pipeline for palette prediction from JPG images. The system maps one image to an unordered set of RGB colors with variable cardinality using a frozen DINOv3 ViT-L/16 backbone, a DETR-style palette query decoder, and Hungarian bipartite matching. The report is written in a scientific-paper style and explains the method step by step, including data validation, preprocessing, feature extraction, mask handling, side-channel color features, loss design, evaluation metrics, inference, and training diagnostics.

1 Problem Definition

The task is to predict an unordered palette from a single JPG image:

$$I \mapsto \mathcal{P} = \{c_1, c_2, \dots, c_K\}, \quad (1)$$

where each color $c_i \in [0, 255]^3$ is an RGB triplet and K is image-dependent. Since palette colors have no semantic ordering, the target is a *set*, not a sequence. Therefore, the learning objective must be permutation invariant with respect to the target row order in the annotation JSON.

The implemented architectural choice is:

JPG image $\rightarrow$ Frozen DINOv3 features $\rightarrow$ Query decoder $\rightarrow$ unordered RGB palette.

2 Data Sources and Pairing

The dataset is formed by two aligned directories:

- images: `E:/End-to-End/Data/jpg`
- palette annotations: `E:/End-to-End/Data/palettes`

Files are paired strictly by zero-padded stem:

$$000000.jpg \leftrightarrow 000000.json. \quad (2)$$

Each annotation is expected to contain:

- `sample_id`
- `num_colors`
- `rgb`
- `hex`

3 Dataset Validation and Filtering

3.1 Validation Rules

The validation stage performs the following checks for every pair:

1. Verify that the file stem is numeric.
2. Verify that `sample_id = int(stem)`.
3. Verify that

$$\text{num_colors} = |\text{rgb}| = |\text{hex}|. \quad (3)$$

4. Verify that every RGB entry contains exactly three integers in $[0, 255]$.
5. Verify that every HEX string matches its RGB counterpart exactly.
6. Detect duplicate RGB values inside the same palette.

Originally, duplicate RGB values were only warnings. In the revised pipeline, duplicate colors are treated as invalid training targets because the target is mathematically a set and repeated elements create ambiguity.

3.2 Training Cardinality Cap

The training set is explicitly restricted to palettes with at most 12 colors:

$$K \leq 12. \quad (4)$$

This policy was introduced after observing extreme outliers with very large palette cardinalities. Using a very large `max_num_colors` would force the model to allocate many unnecessary negative queries and destabilize presence supervision.

Thus, the trainable subset is:

$$\mathcal{D}_{trainable} = \{(I, \mathcal{P}) \mid |\mathcal{P}| \leq 12\}. \quad (5)$$

Samples above this threshold are not silently discarded; they are reported explicitly in the validation output.

4 Preprocessing

4.1 Aspect-Preserving Resize and Letterboxing

The image transform proceeds in four steps:

1. Load the JPG as RGB.
2. Resize the image while preserving aspect ratio so that the long side becomes 512.
3. Place the resized content into a fixed 512×512 square canvas via letterboxing.
4. Normalize the tensor using the exact mean and standard deviation required by `vit116_lvd`.

Let the raw image be $I \in \mathbb{R}^{H \times W \times 3}$. After resize and letterbox, the model receives

$$\tilde{I} \in \mathbb{R}^{512 \times 512 \times 3}. \quad (6)$$

4.2 Valid-Pixel Mask

The preprocessing stage also produces a valid-content mask

$$M_{pix} \in \{0, 1\}^{512 \times 512}, \quad (7)$$

where

$$M_{pix}(x, y) = \begin{cases} 1 & \text{if } (x, y) \text{ belongs to resized real image content,} \\ 0 & \text{if } (x, y) \text{ belongs to letterbox padding.} \end{cases} \quad (8)$$

This mask is critical because padded areas must not be interpreted as valid palette evidence.

4.3 Raw Unnormalized Tensor

Besides the normalized input tensor, the pipeline preserves the raw pre-normalization tensor. This is needed for the chromatic side-channel described later, because exact color statistics should be computed in raw image space rather than from standardized backbone input.

5 Frozen DINOv3 Backbone

The backbone is the official DINOv3 ViT-L/16 LVD model, referenced in the codebase as `vitl16_lvd`. The backbone is frozen:

$$\frac{\partial \mathcal{L}}{\partial \theta_{DINO}} = 0. \quad (9)$$

All training updates are restricted to the palette head.

5.1 Multi-Level Intermediate Features

The feature extractor obtains normalized intermediate patch tokens and the class token using the public DINOv3 API. The selected transformer block indices are

$$[5, 11, 17, 23]. \quad (10)$$

These should be understood as *multi-level blocks*, not literally the last four blocks. The design rationale is that color-sensitive appearance cues may survive better when features are aggregated across depth instead of relying only on the final representation.

For each selected block l , the backbone returns:

$$T^{(l)} \in \mathbb{R}^{N \times C}, \quad \text{CLS}^{(l)} \in \mathbb{R}^C, \quad (11)$$

where N is the number of image patches and C is the hidden dimension of the backbone.

The multi-level patch memory is formed by concatenation:

$$T_{fused} = \text{concat} \left(T^{(5)}, T^{(11)}, T^{(17)}, T^{(23)} \right) \in \mathbb{R}^{N \times 4C}. \quad (12)$$

The final class token is taken from the deepest selected block:

$$\text{CLS}_{final} = \text{CLS}^{(23)}. \quad (13)$$

6 Patch-Space Masking

Because the backbone operates at patch resolution, the pixel mask is converted into a patch mask:

$$M_{patch} \in \{0, 1\}^N. \quad (14)$$

Each patch is marked valid if most of its pixels correspond to true image content. This patch mask is then used as a transformer memory padding mask:

$$\text{memory_key_padding_mask} = \neg M_{patch}. \quad (15)$$

This ensures that letterbox padding does not influence:

- cross-attention from queries to memory,
- pooled count features,
- auxiliary count evidence.

7 Raw-Color Side-Channel

DINOv3 is strong at structural and semantic representation learning, but exact RGB prediction may require more direct access to pixel-level chromatic statistics. Therefore, the implemented pipeline augments the DINO memory with a compact patch-wise raw-color descriptor.

For each valid patch, the side-channel computes:

- mean RGB,
- RGB standard deviation,
- RGB minimum,
- RGB maximum,
- mean OKLab.

Let this descriptor be:

$$R_i \in \mathbb{R}^{d_r} \quad (16)$$

for patch i . Then the final memory token for patch i is formed as:

$$Z_i = W [T_{fused,i} \parallel R_i], \quad (17)$$

where $\parallel$ denotes concatenation and W is a trainable projection.

This preserves the original backbone choice while allowing the head to “read” direct color evidence from the image.

8 Query-Based Palette Decoder

The head is DETR-like and uses a fixed number of learned queries:

$$Q = K_{max} + Q_{spare}. \quad (18)$$

With the current training cap:

$$K_{max} = 12, \quad Q_{spare} = 4, \quad Q = 16. \quad (19)$$

Each query is decoded through a pre-normalized transformer decoder. Each decoded query emits:

- RGB prediction $\hat{c}_q \in [0, 1]^3$,
- presence logit $\hat{p}_q \in \mathbb{R}$.

The model additionally predicts the global palette count:

$$\hat{K} = \arg \max \text{count_logits}. \quad (20)$$

9 Count Prediction

Count prediction is not based only on CLS. Instead, the implemented count branch aggregates three signals:

$$f_{count} = [\text{CLS}_{proj} \parallel \text{MeanPool}(Z, M_{patch}) \parallel \text{AttnPool}(Z, M_{patch})]. \quad (21)$$

This is motivated by the concern that rare colors may weakly affect CLS alone. Masked mean pooling and masked attention pooling provide additional evidence grounded in valid patch memory.

10 Hungarian Matching

Because the target is an unordered set, the model uses one-to-one Hungarian assignment between predicted queries and target colors.

For predicted query q and target color j , the matching cost is:

$$\mathcal{C}_{qj} = \lambda_{rgb} \|\hat{c}_q - c_j\|_1 + \lambda_{perc} d_{OKLab}(\hat{c}_q, c_j) - \lambda_{pres} \log \sigma(\hat{p}_q). \quad (22)$$

The matching is detached from gradient flow and solved with Hungarian bipartite assignment.

11 Loss Function

After matching, the final training loss is composed of several terms.

11.1 Matched RGB Regression

For matched pairs (q, j) :

$$\mathcal{L}_{rgb} = \frac{1}{B} \sum_{b=1}^B \left(\frac{1}{|\mathcal{M}_b|} \sum_{(q,j) \in \mathcal{M}_b} \text{SmoothL1}(\hat{c}_{b,q}, c_{b,j}) \right). \quad (23)$$

The key design detail is that the loss is averaged *inside each image first*, then across the batch. This prevents images with large palettes from dominating optimization.

11.2 Perceptual Color Loss

For the same matched pairs:

$$\mathcal{L}_{perc} = \frac{1}{B} \sum_{b=1}^B \left(\frac{1}{|\mathcal{M}_b|} \sum_{(q,j) \in \mathcal{M}_b} d_{OKLab}(\hat{c}_{b,q}, c_{b,j}) \right). \quad (24)$$

11.3 Presence Loss

Presence supervision is strongly imbalanced because many queries do not correspond to true palette entries. Therefore, the implementation uses a focal-style binary loss:

$$\mathcal{L}_{pres} = \text{FocalBCE}(\hat{p}, y_{pres}). \quad (25)$$

Positive and negative presence losses are logged separately for diagnostics.

11.4 Count Loss

Count is supervised with categorical cross-entropy:

$$\mathcal{L}_{count} = \text{CE}(\text{count_logits}, K). \quad (26)$$

11.5 Count Consistency Loss

The model maintains a second implicit count estimate from the sum of presence probabilities:

$$\hat{K}_{pres} = \sum_q \sigma(\hat{p}_q). \quad (27)$$

Consistency is encouraged with:

$$\mathcal{L}_{cons} = \left| \hat{K}_{pres} - K \right|. \quad (28)$$

11.6 Auxiliary Decoder Losses

Intermediate decoder layers can also produce RGB and presence outputs. Auxiliary supervision is applied to those intermediate predictions, but count is not duplicated at every layer.

11.7 Total Loss

The final loss is:

$$\mathcal{L} = \lambda_{rgb} \mathcal{L}_{rgb} + \lambda_{perc} \mathcal{L}_{perc} + \lambda_{pres} \mathcal{L}_{pres} + \lambda_{count} \mathcal{L}_{count} + \lambda_{cons} \mathcal{L}_{cons} + \lambda_{aux} \mathcal{L}_{aux}. \quad (29)$$

12 Evaluation Metrics

All evaluation is Hungarian-aligned and permutation invariant. Reported metrics include:

- count accuracy,
- count MAE,
- matched RGB MAE,
- $\Delta E00$ mean, median, P90, P95,
- percentages with $\Delta E00 \leq 1, 2, 5$,
- exact-cardinality rate,
- palette success rate,
- false-positive query rate,
- false-negative color rate.

12.1 Count Agreement Metrics

Two count pathways are compared explicitly:

$$\hat{K}_{head} = \arg \max \text{count_logits}, \quad \hat{K}_{pres} = \text{round} \left(\sum_q \sigma(\hat{p}_q) \right). \quad (30)$$

The pipeline reports:

- count-head accuracy,
- presence-derived count accuracy,
- absolute disagreement $|\hat{K}_{head} - \hat{K}_{pres}|$.

12.2 Unified Set Error

To avoid selecting checkpoints based only on matched-color quality, a unified set-level error is used:

$$E_{set} = \frac{\sum_i \min(\Delta E_{00i}, c) + c|K - \hat{K}|}{\max(K, \hat{K})}, \quad (31)$$

with default penalty constant $c = 10$.

This metric jointly reflects color quality and cardinality correctness and is used for:

- best checkpoint selection,
- early stopping,
- model comparison.

13 Training Loop

The training loop uses:

- AdamW,
- warmup plus cosine decay,
- automatic mixed precision,
- gradient clipping,
- optional gradient accumulation,
- deterministic seeding,
- checkpoint save/resume.

Extensive progress tracking was added through tqdm bars and epoch-level summaries.

14 Early Stopping

Early stopping was made deliberately more conservative by introducing:

- larger patience,
- a minimum number of epochs before patience is consumed,
- a minimum improvement threshold δ .

This reduces the chance of terminating training because of minor metric fluctuations.

15 Inference

At inference time, the model:

1. predicts count logits,
2. ranks queries by presence probability,
3. keeps the highest-confidence unique RGB colors,
4. rounds them deterministically to 8-bit RGB,
5. serializes them with exact uppercase HEX values.

A lightweight UI was added for checkpoint loading, image upload, palette-swatch visualization, and JSON inspection.

16 Testing

The current codebase includes focused tests for:

- dataset pairing and validation,
- collation,
- head output shape,
- Hungarian matching,
- permutation invariance,
- presence supervision,
- count loss,
- patch-mask effect,
- unified set error,
- JSON formatting,
- checkpoint round-trip,
- preprocessing shape guarantees.

At the latest check, the suite status was:

17 passed, 2 skipped. (32)

17 Current Status and Remaining Empirical Work

The code path for the revised pipeline has been implemented, but a few empirical results still need to be regenerated after the latest changes:

- full preflight report on the real dataset,
- actual tiny-overfit results on the revised architecture,
- refreshed quantitative report with current runtime metrics.

18 Conclusion

The implemented method now provides a stronger and more principled foundation for palette prediction than the initial version. The main improvements are:

- trainable-set filtering with explicit reporting,
- mask-aware letterbox handling,
- raw-color side-channel fused with frozen DINO features,
- more stable matching and presence supervision,
- set-level checkpoint metric,
- stronger diagnostics and testing infrastructure.

In summary, the system remains faithful to the original high-level design — frozen DINOv3 plus DETR-style set decoding — while significantly improving robustness in data handling, optimization stability, and evaluation quality.